

CS2300: File Structures and Introduction to Database Systems

Lecture 10: SQL Queries

San Yeung

Query Components

- A query can contain the following **clauses**
 - **Select**
 - **From**
 - **Where**
 - **Group by**
 - **Having**
 - **Order by**
- Only **Select** and **From** are **obligatory**
- Order of clauses is always as above

Very Basic SQL Query

```
SELECT [Distinct] Attributes  
FROM relations
```

- One basic statement for retrieving information from a database
- *Attributes*: The attributes which will appear in the query result (For example: id, name).
- *Relations*: Relations to perform the query on.
- *DISTINCT*: Optional keyword to delete duplicates

Tables as Sets in SQL

- SQL does not automatically eliminate duplicate tuples in query results
- For aggregate operations duplicates must be accounted for
- Use the keyword **DISTINCT** in the `SELECT` clause
 - Only distinct tuples should remain in the result

Query 11. Retrieve the salary of every employee (Q11) and all distinct salary values (Q11A).

Q11: **SELECT** **ALL** Salary
 FROM **EMPLOYEE;**

Q11A: **SELECT** **DISTINCT** Salary
 FROM **EMPLOYEE;**

Example

StudentID	StudentDept	StudentName	StudentAge
1123	Math	Moshe	25
2245	Computers	Mickey	26
55611	Math	Menahem	29

Select studentID, studentName
From students

StudentID	StudentName
1123	Moshe
2245	Mickey
55611	Menahem

Basic SQL Query

```
SELECT [Distinct] Attributes  
FROM relations  
WHERE condition
```

- Logical comparison operators
 - =, <, <=, >, >=, and <>
- *condition*: A Boolean condition (For example: Eid>21, or Ename='Yuval'). Only tuples which return 'true' for this condition will appear in the result.
 - Selection conditions include join conditions when multiple relations are involved

Example

StudentID	StudentDept	StudentName	StudentAge
1123	Math	Moshe	20
2245	NULL	Mickey	18
55611	Math	Menahem	21

Select studentID, studentName
From students
Where StudentDept='Math'

StudentID	StudentName
1123	Moshe
55611	Menahem

Example

Logical components: AND, OR

Null verification: IS NULL, IS NOT NULL

StudentID	StudentDept	StudentName	StudentAge
1123	Math	Moshe	20
2245	NULL	Mickey	18
55611	Math	Menahem	21

SELECT StudentName

FROM Students

WHERE age>=17 **AND** StudentDept **IS NOT NULL** ;

Example

Checking against a list with **IN, NOT IN**

StudentID	StudentDept	StudentName	StudentAge
1123	Math	Moshe	20
2245	NULL	Mickey	18
55611	Math	Bob	21

```
SELECT StudentID, StudentName  
FROM Students  
WHERE StudentID IN (1123,2245);
```

Example

BETWEEN comparison operator

StudentID	StudentDept	StudentName	StudentAge
1123	Math	Moshe	20
2245	NULL	Mickey	18
55611	Math	Bob	21

```
SELECT StudentID, StudentName
FROM   Students
WHERE StudentID IN (1123,2245) OR
        StudentName BETWEEN('Alice' and 'Karl');
```

The LIKE Operator: Substring Pattern Matching

- A pattern matching operator (regular expression)
- Basic format: **colname LIKE pattern**
 - % replaces an arbitrary number of zero or more characters
 - underscore () replaces a single character
- Example:

```
SELECT StudentName  
FROM Students  
WHERE StudentName LIKE 'S%n%a';
```

Ordering of Query Results

- Use **ORDER BY** clause
 - Keyword **DESC** to see result in a descending order of values
 - Keyword **ASC** to specify ascending order explicitly
 - Multiple columns: which attribute would be ordered *first*?

```
SELECT    <attribute list>
FROM      <table list>
[ WHERE   <condition> ]
[ ORDER BY <attribute list> ];
```

```
ORDER BY D.Dname DESC, E.Lname
ASC, E.Fname ASC
```

Ordering of Query Results

```
ORDER BY D.Dname DESC, E.Lname  
ASC, E.Fname ASC
```

Department Name	Employee Last Name	Employee First Name
Technology	Adams	Aaron
Technology	Adams	Bethany
Sales	Carter	Zoe
Sales	Carter	Anna
Research	Doe	John
Research	Doe	Alice

Unspecified WHERE Clause and Use of the Asterisk

- Missing WHERE clause
 - Indicates no condition on tuple selection
- Effect is a CROSS PRODUCT
 - Result is all possible tuple combinations result

Queries 9 and 10. Select all EMPLOYEE Ssns (Q9) and all combinations of EMPLOYEE Ssn and DEPARTMENT Dname (Q10) in the database.

Q9: SELECT Ssn
 FROM EMPLOYEE;

Q10: SELECT Ssn, Dname
 FROM EMPLOYEE, DEPARTMENT;

- Specify an asterisk (*)
 - Retrieve all the attribute values of the selected tuples
 - The * can be prefixed by the relation name; e.g., EMPLOYEE *

Q1C: **SELECT** *

FROM EMPLOYEE

WHERE Dno=5;

Q1D: **SELECT** *

FROM EMPLOYEE, DEPARTMENT

WHERE Dname='Research' **AND** Dno=Dnumber;

Q10A: **SELECT** *

FROM EMPLOYEE, DEPARTMENT;

Ambiguous Attribute Names

- Same name can be used for two (or more) attributes in different relations
 - Must **qualify** the attribute name with the relation name to prevent ambiguity
- **Aliases or tuple variables**
 - Declare alternative relation names

Query 8. For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn=S.Ssn;
```

Aliasing, Renaming and Tuple Variables (contd.)

- The attribute names can also be renamed

```
EMPLOYEE AS E(Fn, Mi, Ln, Ssn,  
Bd, Addr, Sex, Sal, Sssn, Dno)
```

- Note that the relation EMPLOYEE now has a variable name E which corresponds to a tuple variable
- The “AS” may be dropped in most SQL implementations

Arithmetic Operations

- Standard arithmetic operators:
 - Addition (+), subtraction (–), multiplication (*), and division (/) may be included as a part of **SELECT**

Query 13. Show the resulting salaries if every employee working on the 'ProductX' project is given a 10 percent raise.

```
SELECT      E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal
FROM        EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
WHERE       E.Ssn=W.Essn AND W.Pno=P.Pnumber AND
              P.Pname='ProductX';
```

Basic SQL Query

```
SELECT [Distinct] attributes  
FROM relations  
WHERE condition;
```

Important! The evaluation order is:

1. Compute the cross product of the tables in *relations*.
2. Delete all rows that do not satisfy *condition*.
3. Delete all columns that do not appear in *attributes*.
4. If *Distinct* is specified, eliminate duplicate rows.

Basic SQL Query

- If more than one relation is mentioned, the operation is on the **Cartesian product** between the relations
- Example:

```
Select sname  
From sailor, reserve  
Where sailor.sid=reserve.sid
```

Stage 1: Sailor x Reserve

Sailor				Reserve		
sid	sname	rating	age	sid	bid	day
22	Dustin	7	45.0	22	101	10/10/96
22	Dustin	7	45.0	58	103	11/12/96
31	Lubber	8	55.5	22	101	10/10/96
31	Lubber	8	55.5	58	103	11/12/96
58	Rusty	10	35.0	22	101	10/10/96
58	Rusty	10	35.0	58	103	11/12/96

Stage 2: “where sailor.sid=reserve.sid”

Sailor				Reserve		
sid	sname	rating	age	sid	bid	day
22	Dustin	7	45.0	22	101	10/10/96
22	Dustin	7	45.0	58	103	11/12/96
31	Lubber	8	55.5	22	101	10/10/96
31	Lubber	8	55.5	58	103	11/12/96
58	Rusty	10	35.0	22	101	10/10/96
58	Rusty	10	35.0	58	103	11/12/96

Stage 2: “where sailor.sid=reserve.sid”

Sailor				Reserve		
sid	sname	rating	age	sid	bid	day
22	Dustin	7	45.0	22	101	10/10/96
58	Rusty	10	35.0	58	103	11/12/96

Stage 3: “select sname”

Sailor				Reserve		
sid	sname	rating	age	sid	bid	day
22	Dustin	7	45.0	22	101	10/10/96
58	Rusty	10	35.0	58	103	11/12/96

Stage 3: “select sname”

Final answer

Sname
Dustin
Rusty

Three SET Operators

- [SQL Query] UNION [SQL Query]
- [SQL Query] EXCEPT [SQL Query]
- [SQL Query] INTERSECT [SQL Query]
- Note: The operators remove duplicates by default!
 - Alternatives: UNION ALL, EXCEPT ALL, INTERSECT ALL

Sailors who've reserved a red or green boat

```
SELECT S.sname -- Should S.sid be included?
      FROM Sailors S, Boats B, Reserves R
     WHERE S.sid = R.sid and R.bid = B.bid
           and B.color = 'red'

UNION

SELECT S.sname
      FROM Sailors S, Boats B, Reserves R
     WHERE S.sid = R.sid and R.bid = B.bid
           and B.color = 'green' ;
```

Sailors who've reserved red **AND** green boats

```
SELECT S.sid, S.sname
      FROM Sailors S, Boats B, Reserves R
     WHERE S.sid = R.sid and R.bid = B.bid
           and B.color = 'red'
```

INTERSECT

```
SELECT S.sid, S.sname
      FROM Sailors S, Boats B, Reserves R
     WHERE S.sid = R.sid and R.bid = B.bid
           and B.color = 'green' ;
```

What if we remove "S.sid"?

Sailors who've reserved a red boat
but did not reserve any green boat

```
SELECT S.sid, S.sname
      FROM Sailors S, Boats B, Reserves R
     WHERE S.sid = R.sid and R.bid = B.bid
           and B.color = 'red'
```

Minus

```
SELECT S.sid, S.sname
      FROM Sailors S, Boats B, Reserves R
     WHERE S.sid = R.sid and R.bid = B.bid
           and B.color = 'green' ;
```

Aggregate Operators

- The aggregate operators available in SQL:
 - **COUNT (*)** : count number of tuples
 - **COUNT ([DISTINCT] A)** : count number of not-null values of attribute A
 - **SUM ([DISTINCT] A)**
 - **AVG ([DISTINCT] A)**
 - **MAX (A)**
 - **MIN (A)**

Examples

```
SELECT COUNT(*)  
  FROM Sailors S;
```



```
SELECT COUNT(sid)  
  FROM Sailors S;
```

```
SELECT COUNT(DISTINCT color)  
  FROM Boats;
```

Example

- Find the age of the oldest sailor

```
SELECT Max(S.age)
FROM Sailor S;
```

How to find the name of the oldest sailor?

Grab individual tuples

Aggregation

```
SELECT S.sname, MAX(S.age)
FROM Sailors S;
```

Problematic in most SQL databases.



Question

- Find name and age of the oldest Sailor?

```
SELECT S.sname, MAX(S.age)
FROM Sailors S;
```

Wrong!

```
SELECT S.sname, S.age
FROM Sailors S
WHERE S.age =
      (SELECT MAX(S2.age)
       FROM Sailors S2);
```

Correct!

Find Average Age for **each** Rating?

- So far, aggregation has been applied to all tuples that passed the WHERE clause test.
- How can we apply aggregation to *groups* of tuples defined by some attributes?

Example

- Find Average Age *for each* Rating

```
SELECT AVG(age)
FROM Sailors
GROUP BY rating;
```

Example

Sailors

<u>sid</u>	Sname	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.5
58	Rusty	10	35.0
63	Fluffy	7	44.0
78	Morley	7	31.0
84	Popeye	10	33.0

```
SELECT AVG(age)
FROM Sailors
GROUP BY rating;
```

Example

Sailors

<u>sid</u>	sname	rating	age
22	Dustin	7	45.0
63	Fluffy	7	44.0
78	Morley	7	31.0
31	Lubber	8	55.5
58	Rusty	10	35.0
84	Popeye	10	33.0

```
SELECT rating,AVG(age)
FROM Sailors
GROUP BY rating;
```

rating	Avg_age
7	40
8	55.5
10	34

HAVING Clause

```
SELECT [Distinct] attributes  
      FROM relation-list  
      WHERE condition  
      GROUP BY grouping-attributes  
      HAVING group-condition;
```

- **attributes**: must appear in *grouping-attributes* or are aggregation operators
- *group-condition*: Constrains groups. Can only constrain attributes appearing in *grouping-attributes* or aggregation operators

Evaluation Order

```
SELECT [Distinct] attributes
FROM relation-list
WHERE condition
GROUP BY grouping-attributes
HAVING group-condition;
```

1. Compute cross product of *relation-list*
2. Tuples failing *condition* are thrown away
3. Tuples are partitioned into groups by values of *grouping-attributes*
4. The *group-condition* is applied to eliminate groups
5. **One answer is generated for each group!**

```
SELECT AVG(age)
FROM Sailors
Where age<50
GROUP BY rating
Having count(*)>2;
```

Step 1

<u>Sid</u>	Sname	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.5
58	Rusty	10	35.0
63	Fluffy	7	44.0
78	Morley	7	31.0
84	Popeye	10	33.0

<u>Sid</u>	Sname	rating	age
22	Dustin	7	45.0
58	Rusty	10	35.0
63	Fluffy	7	44.0
78	Morley	7	31.0
84	Popeye	10	33.0

```
SELECT AVG(age)
FROM Sailors
Where age<50
GROUP BY rating
Having count(*)>2;
```

Step 2

<u>Sid</u>	sname	rating	age
22	Dustin	7	45.0
63	Fluffy	7	44.0
78	Morley	7	31.0
58	Rusty	10	35.0
84	Popeye	10	33.0

```
SELECT AVG(age)
FROM Sailors
Where age<50
GROUP BY rating
Having count(*)>2;
```

Step 3

<u>Sid</u>	sname	rating	age
22	Dustin	7	45.0
63	Fluffy	7	44.0
78	Morley	7	31.0
58	Rusty	10	35.0
84	Popeye	10	33.0

<u>Sid</u>	sname	rating	age
22	Dustin	7	45.0
63	Fluffy	7	44.0
78	Morley	7	31.0

```
SELECT AVG(age)
FROM Sailors
Where age<50
GROUP BY rating
Having count(*)>2;
```

Step 4

<u>Sid</u>	sname	rating	age
22	Dustin	7	45.0
63	Fluffy	7	44.0
78	Morley	7	31.0



Final Answer:

40

```
SELECT AVG(age)
FROM Sailors
Where age<50
GROUP BY rating
Having count(*)>2 AND Sid>10;
```

Still correct?

<u>Sid</u>	sname	rating	age
22	Dustin	7	45.0
63	Fluffy	7	44.0
78	Morley	7	31.0
58	Rusty	10	35.0
84	Popeye	10	33.0

Aggregate Operators

- **NULL** values are ignored during calculation
- Can be used in Select, Having or Order By clause. **Can't be used in WHERE clause!**
 - Because aggregation applies to all tuples satisfying ***conditions*** in WHERE clause
- Can't be placed together with regular attributes in Select clause, unless the attribute appears in **Group By** clause.

SQL Queries

```
SELECT Attributes  
FROM relations  
WHERE condition  
Group by grouping-attributes  
Having group-condition  
Order by Attribute ASC, Attribute DESC...;
```

*If you want to order by attributes, they MUST be included in the '**SELECT**' clause either as grouping attributes or an aggregated value.*

```
SELECT AVG(age)
FROM Sailors
Where age<50
GROUP BY rating
Having count(*)>1
ORDER BY sname;
```

Still correct?

<u>Sid</u>	sname	rating	age
22	Dustin	7	45.0
63	Fluffy	7	44.0
78	Morley	7	31.0
58	Rusty	10	35.0
84	Popeye	10	33.0

```
SELECT rating, AVG(age)
FROM Sailors
Where age<50
GROUP BY rating
Having count(*)>1
ORDER BY rating;
```

Still correct?

<u>Sid</u>	sname	rating	age
22	Dustin	7	45.0
63	Fluffy	7	44.0
78	Morley	7	31.0
58	Rusty	10	35.0
84	Popeye	10	33.0

```
SELECT rating, AVG(age), MIN(age)
FROM Sailors
Where age<50
GROUP BY rating
Having count(*)>1
ORDER BY MIN(age);
```

Still correct?

<u>Sid</u>	sname	rating	age
22	Dustin	7	45.0
63	Fluffy	7	44.0
78	Morley	7	31.0
58	Rusty	10	35.0
84	Popeye	10	33.0

Nested Queries

Nested Queries

- A complete SELECT query, called (*nested query* or *inner query*), can be specified within the WHERE-clause of another query, called the *outer query*
- In general, we can have several levels of nested queries
- Many of the previous queries can be specified in an alternative form using nesting

Example

- Sailor(sid, sname)
- Reserve(sid, bid, day)

Find the IDs of boats reserved by sailor George.

```
Select R.bid  
From Reserves R, Sailor S  
Where R.sid = S.sid and sname='George' ;
```

This query can also be specified by using a nested query as shown in the next slide.

Nested Queries in WHERE

- Equality nested query:

Select R.bid

From Reserves R

Where R.sid = (**Select sid From Sailor
Where sname='George'**) ;

Nested query

Step 1: The inner query selects sid of the sailor whose name is 'George'.
Step 2: The outer query selects boat ID if the corresponding sailor's ID is equal to the result of the inner query.

This works only when the subquery returns exactly one tuple

$$\pi_{R.bid}(\sigma_{R.sid=(\pi_{sid}(\sigma_{sname='George'}Sailor))} R)$$

Nested Queries in WHERE

- Subqueries with **multiple** results:

```
SELECT S.sname
FROM Sailors S
WHERE S.sid IN (SELECT R.sid
                FROM Reserves R
                WHERE R.bid = 103);
```

Result: Names of sailors who reserved boat 103

Q: What would happen if we wrote **NOT IN**?

A: We would get names of sailors who did not reserve boat 103 !

Set Comparison

```
SELECT *  
  FROM Sailors S1  
 WHERE S1.age > ANY (SELECT S2.age  
                      FROM Sailors S2) ;
```

Sailors who are not the youngest

We can also use *`op ALL'* (*op* is $>$, $<$, $=$, $\geq$, $\leq$, or $<>$)

What happens if we use **ALL** in the above query?

What does this produce?

```
SELECT S.sname
FROM Sailors S      step3
WHERE S.sid NOT IN
  (SELECT R.sid
   FROM Reserves R
   WHERE R.bid IN
     (SELECT B.bid
      FROM Boats B
      WHERE B.color='red'))
```

step2 {

step1 }

What does this produce?

```
SELECT S.sname
FROM Sailors S      step3
WHERE S.sid NOT IN
  (SELECT R.sid
   FROM Reserves R
   WHERE R.bid IN
     (SELECT B.bid
      FROM Boats B
      WHERE B.color='red'))
```

Diagram illustrating the steps of the SQL query:

- step1** (red bracket): The innermost subquery: `(SELECT B.bid FROM Boats B WHERE B.color='red'))`
- step2** (blue bracket): The middle subquery: `(SELECT R.sid FROM Reserves R WHERE R.bid IN (SELECT B.bid FROM Boats B WHERE B.color='red'))`
- step3** (green text): The main query: `SELECT S.sname FROM Sailors S WHERE S.sid NOT IN (SELECT R.sid FROM Reserves R WHERE R.bid IN (SELECT B.bid FROM Boats B WHERE B.color='red'))`

Names of sailors who did not reserve a red boat

Correlated Nested Queries

- If a condition in the WHERE-clause of a *nested query* references an attribute of a relation declared in the *outer query*, the two queries are said to be **correlated**.
- The result of a correlated nested query is *different for each tuple (or combination of tuples) of the relation(s) in the outer query*
 - Take tuples from outer query as input of correlated nested query
 - A correlated query is executed repeatedly (also known as a repeating nested query)
 - Once for each row evaluated by the outer query

Example

EMPLOYEE(SSN, FNAME, LNAME)
DEPENDENT(ESSN, NAME)

Retrieve the names of employees who have at least one dependent.

```
SELECT      FNAME, LNAME
FROM        EMPLOYEE
WHERE       EXISTS (SELECT *
                    FROM  DEPENDENT
                    WHERE  SSN=ESSN)
```

For-loop behavior: for every employee, determines if he/she has at least one dependent

Exists and Not Exists

- **Exists R :**
 - If R is not empty, returns true.
 - In the correlated nested query:
 - For every tuple in the outer loop, the inner loop is tested.
 - If the result of the inner loop is not empty, the outer tuple is added to the final result.
- **Not Exists R :**
 - If R is empty, returns true.
 - In the correlated nested query:
 - For every tuple in the outer loop, the inner loop is tested.
 - If the result of the inner loop is empty, the outer tuple is added to the final result.

Example

Retrieve the names of employees who have at least one dependent.

```

SELECT      Ssn, FNAME, LNAME
FROM        EMPLOYEE AS E
WHERE       EXISTS (SELECT *
                    FROM  DEPENDENT
                    WHERE  E.SSN=ESSN)
  
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Example

EMPLOYEE(SSN, FNAME, LNAME)
DEPENDENT(ESSN, NAME)

Retrieve the names of employees who have **no** dependent.

```
SELECT      FNAME, LNAME
FROM        EMPLOYEE
WHERE       NOT EXISTS (SELECT *
                        FROM DEPENDENT
                        WHERE SSN=ESSN)
```

Correlated Nested Queries

- CNQ: a subquery that uses values from the outer query
- The subquery (inner query) is evaluated _____ for each row processed by the outer query.
- *Application:* Used in cases where each row from the outer query needs to be **evaluated** with the data from the inner query.

Non-Correlated Nested Queries

- Subqueries can be executed **independently**.
- Is it possible to replace a CNQ with a non-correlated query?

Example

EMPLOYEE(SSN, FNAME, LNAME)
DEPENDENT(ESSN, NAME)

Retrieve the names of employees who have at least one dependent.

```
SELECT      DISTINCT SSN, FNAME, LNAME
FROM        EMPLOYEE, DEPENDENT
WHERE       SSN = ESSN;
```

-- Remember: DISTINCT applies to the entire set of columns listed

In some cases, it might be challenging or even impossible to rewrite some CNQ without losing clarity or increasing complexity significantly.

(readability, maintainability, and performance)

Example

EMPLOYEE(SSN, FNAME, LNAME)

DEPENDENT(ESSN, NAME)

DEPARTMENT(DEPT#, MGR_SSN)

Retrieve the names of managers who have at least one dependent.

```

SELECT      FNAME,  LNAME
FROM        EMPLOYEE
WHERE       EXISTS ( SELECT      *
                      FROM        DEPENDENT
                      WHERE       SSN=ESSN)
            AND EXISTS ( SELECT      *
                      FROM        DEPARTMENT
                      WHERE       SSN = MGR_SSN) ;
    
```

Can you rewrite this query using only a single nested query or no nested query?

Sailors who reserved all boats (Division)

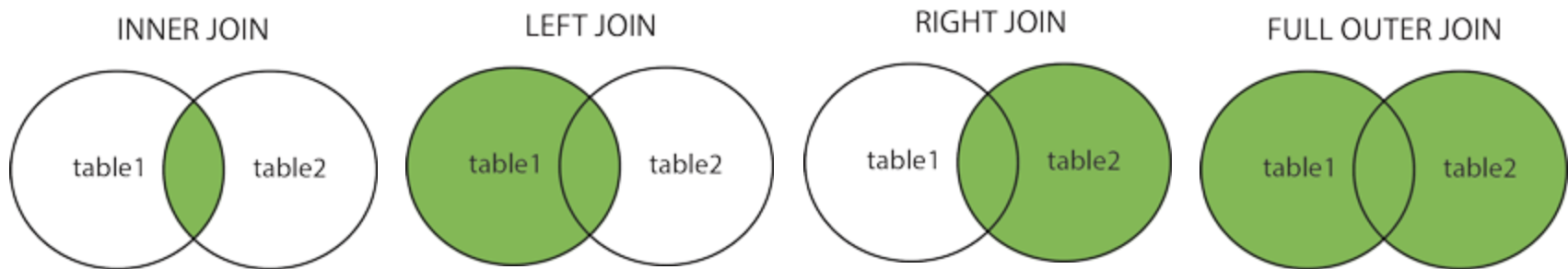
```
SELECT S.sid
  FROM Sailors S
 WHERE NOT EXISTS ((SELECT B.bid
                     FROM Boats B)
                   MINUS
                   (SELECT R.bid
                     FROM Reserves R
                     WHERE R.sid = S.sid)) ;
```

Sailors who reserved all boats (Approach 2)

```
SELECT sid
FROM Sailors S
WHERE NOT EXISTS
    (SELECT B.bid
     FROM Boats B
     WHERE B.bid NOT IN
         (SELECT R.bid
          FROM Reserves R
          WHERE R.sid = S.sid) ) ;
```

SQL Joins

- **(INNER) JOIN:** Returns records that have matching values in both tables
- **LEFT (OUTER) JOIN:** Return all records from the left table, and the matched records from the right table
- **RIGHT (OUTER) JOIN:** Return all records from the right table, and the matched records from the left table
- **FULL (OUTER) JOIN:** Return all records when there is a match in either left or right table



SQL Join Syntax

- **SELECT** column-names
- **FROM** table-name1 **JOIN** table-name2
 ON column-name1 = column-name2
- **WHERE** condition

SQL Joins

- `SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID =
Customers.CustomerID;`
- `SELECT Orders.OrderID, Customers.CustomerName,
Shippers.ShipperName
FROM ((Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID)
INNER JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID);`

Use of CASE

- SQL also has a CASE construct
- Used when a value can be different based on certain conditions.
- Can be used in **any** part of an SQL query where *a value is expected*
- Applicable when querying, inserting or updating tuples

EXAMPLE of use of CASE

- The following example shows that employees are receiving different raises in different departments (A variation of the update U6)

- | | | |
|--------|--------------|--------------------|
| UPDATE | EMPLOYEE | |
| SET | Salary = | |
| CASE | WHEN Dno = 5 | THEN Salary + 2000 |
| | WHEN Dno = 4 | THEN Salary + 1500 |
| | WHEN Dno = 1 | THEN Salary + 3000 |

which department do you want to be in? really?

Specifying General Constraints as Assertions in SQL

- **CREATE ASSERTION**

- Specify a query that selects any tuples that violate the desired condition
- constraint: *the salary of an employee must not be greater than the salary of the manager of the department that the employee works for*

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK ( NOT EXISTS ( SELECT *
                     FROM   EMPLOYEE E, EMPLOYEE M,
                     DEPARTMENT D
                     WHERE  E.Salary>M.Salary
                           AND E.Dno=D.Dnumber
                           AND D.Mgr_ssn=M.Ssn ) );
```

Specifying General Constraints as Assertions in SQL

- **CREATE ASSERTION**

- Assertions must always be **true** in the database
- Use only in cases where it goes beyond a simple `CHECK` which applies to individual attributes and domains
- Assertions are checked:
 - At the end of each **transaction** (a sequence of related SQL operations)
 - During **data modification operations** (INSERT, UPDATE, DELETE)
 - Or on demand (**manual** checking)

Relational Views in SQL

A view is a single *virtual table* that is derived from other tables

- The other tables could be base tables or previously defined views
- A view **does not necessarily exist in physical form**, which limits the possible update operations that can be applied to views
- There are no limitations on querying a view
- The **CREATE VIEW** command is used to specify a view by specifying a (virtual) table name and a defining query
- The view attribute names can be *inherited from* the attribute names of the tables in the defining query

Example View

```
CREATE VIEW   WORKS_ON1 AS  
      SELECT FNAME, LNAME, PNAME, HOURS  
      FROM   EMPLOYEE, PROJECT, WORKS_ON  
      WHERE  SSN=ESSN AND PNO=PNUMBER ;
```

```
SELECT   Fname, Lname  
FROM     WORKS_ON1  
WHERE     Pname = 'ProductX' ;
```

Example View

- Once a View is defined, SQL queries can use the View relation in the FROM clause
- View is always up-to-date
 - Responsibility of the DBMS and not the user
- **DROP VIEW** command
 - Dispose of a view

View Implementation, View Update, and Inline Views

- **Strategy1: Query modification** approach
 - Compute the view as and when needed. Do not store permanently
 - Modify view query into a query on underlying base tables
 - Disadvantage: inefficient for views defined via complex queries that are time-consuming to execute

View Implementation, View Update, and Inline Views

- **Strategy 2: View materialization**
 - Physically create a temporary view table when the view is first queried
 - Keep that table on the assumption that other queries on the view will follow
 - Requires efficient strategy for automatically updating the view table when the base tables are updated
- **Incremental update strategy for materialized views**
 - DBMS determines what new tuples must be inserted, deleted, or modified in a materialized view table

- Multiple ways to handle materialization:
 - **immediate update** strategy updates a view as soon as the base tables are changed
 - **lazy update** strategy updates the view when needed by a view query
 - **periodic update** strategy updates the view periodically (in the latter strategy, a view query may get a result that is not up-to-date). This is commonly used in Banks, Retail store operations, etc.

Table 7.2 Summary of SQL Syntax

CREATE TABLE <table name> (<column name> <column type> [<attribute constraint>] { , <column name> <column type> [<attribute constraint>] } [<table constraint> { , <table constraint> }])
DROP TABLE <table name>
ALTER TABLE <table name> ADD <column name> <column type>
SELECT [DISTINCT] <attribute list> FROM (<table name> { <alias> } <joined table>) { , (<table name> { <alias> } <joined table>) } [WHERE <condition>] [GROUP BY <grouping attributes> [HAVING <group selection condition>]] [ORDER BY <column name> [<order>] { , <column name> [<order>] }]
<attribute list> ::= (* (<column name> <function> (([DISTINCT] <column name> *))) { , (<column name> <function> (([DISTINCT] <column name> *))) })
<grouping attributes> ::= <column name> { , <column name> }
<order> ::= (ASC DESC)
INSERT INTO <table name> [(<column name> { , <column name> })] (VALUES (<constant value> , { <constant value> }) { , (<constant value> { , <constant value> }) }) <select statement>)

Table 7.2 Summary of SQL Syntax

DELETE FROM <table name>

[WHERE <selection condition>]

UPDATE <table name>

SET <column name> = <value expression> { , <column name> = <value expression> }

[WHERE <selection condition>]

CREATE [UNIQUE] INDEX <index name>

ON <table name> (<column name> [<order>] { , <column name> [<order>] })

[CLUSTER]

DROP INDEX <index name>

CREATE VIEW <view name> [(<column name> { , <column name> })]

AS <select statement>

DROP VIEW <view name>

NOTE: The commands for creating and dropping indexes are not part of standard SQL.